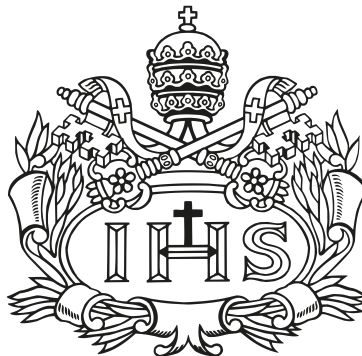


PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
2026



Pontificia Universidad
JAVERIANA
Colombia

Sistema Concurrente de Monitoreo de Mediciones Meteorológicas
Comunicación mediante pipe nominal, hilos POSIX y buffer compartido

Autores

Marlon Jhoan Garcia Restrepo
Juan Felipe González Barón
María Alejandra Rodriguez Betancout

Materia

Sistemas Operativos

Profesor

John Corredor Franco

Fecha de entrega

Mayo de 2026

Índice

1. Resumen	3
2. Introducción	3
3. Objetivos	3
3.1. Objetivo general	3
3.2. Objetivos específicos	4
4. Estructura del proyecto	4
5. Diseño general de la solución	5
5.1. Flujo general del sistema	5
6. Formato de los archivos de entrada	6
7. Estructura de datos principal	6
8. Comunicación mediante pipe nominal	6
8.1. Creación del pipe nominal	7
9. Implementación del Agente	7
9.1. Lectura automática desde la carpeta docs	8
9.2. Envío de lecturas por pipe	8
10. Sincronización entre agentes con semáforo nombrado	9
11. Implementación del Monitor con hilos	9
11.1. Hilos del Monitor	10
12. Buffer compartido productor-consumidor	10
12.1. Inserción en el buffer	11
12.2. Extracción del buffer	11
13. Procesamiento de mediciones	12
14. Cálculo de estadísticas	12
15. Reglas de categorización meteorológica	13
16. Archivo consolidado	14
17. Compilación con Makefile	14

18.Ejecución del sistema	16
18.1. Ejecución del Monitor	16
18.2. Ejecución de un Agente	16
18.3. Ejecución concurrente de varios agentes	16
19.Pruebas realizadas y validación del sistema	16
19.1. Archivos de prueba	17
19.2. Caso de prueba 1: clasificación lluviosa	17
19.3. Caso de prueba 2: clasificación fresca	18
19.4. Caso de prueba 3: archivo mixto	19
19.5. Caso de prueba 4: lecturas sin categoría	19
19.6. Caso de prueba 5: concurrencia con varios agentes	20
19.7. Resumen de validación	20
20.Resultados esperados	21
21.Problemas encontrados y decisiones de diseño	21
21.1. Lectura del pipe con read	21
21.2. Rutas relativas hacia docs	21
21.3. Categorización mediante promedios	22
21.4. Uso de hilos en el Monitor	22
22.Conclusiones	22
23.Referencias	22

1. Resumen

El presente proyecto desarrolla un sistema concurrente en lenguaje C para el procesamiento de mediciones meteorológicas provenientes de diferentes estaciones. La solución está compuesta por dos programas principales: un **Monitor**, encargado de recibir, almacenar temporalmente y procesar las mediciones, y uno o varios **Agentes**, encargados de leer archivos `.cvs` y enviar sus lecturas al Monitor mediante un **pipe nominal**.

El Monitor implementa concurrencia mediante *POSIX threads*. En particular, utiliza un **Hilo Recolector**, que lee las mediciones provenientes del pipe nominal y las introduce en un buffer compartido, y un **Hilo Procesador**, que retira las mediciones del buffer para consolidarlas, calcular estadísticas y clasificar las condiciones meteorológicas. El tamaño del buffer es configurable mediante la bandera `-b`, de acuerdo con el enunciado del proyecto.

Adicionalmente, el sistema permite ejecutar múltiples agentes de forma concurrente. Para evitar conflictos durante la escritura al pipe nominal, los agentes emplean un semáforo nombrado `POSIX`. El resultado del procesamiento se almacena en un archivo consolidado y las estadísticas se imprimen por terminal.

2. Introducción

En los sistemas operativos, la comunicación entre procesos y la sincronización de tareas concurrentes son conceptos fundamentales para construir soluciones capaces de coordinar múltiples unidades de ejecución. Este proyecto aplica dichos conceptos en un escenario práctico: la recepción y procesamiento de mediciones meteorológicas enviadas por varios procesos agentes hacia un proceso monitor.

El problema consiste en permitir que diferentes agentes, ejecutados como procesos independientes, lean archivos de mediciones y envíen sus datos a un monitor común. El monitor debe recibir dichas lecturas por medio de un pipe nominal, almacenarlas temporalmente en un buffer de tamaño limitado y procesarlas de forma concurrente mediante hilos.

La solución propuesta integra varios conceptos propios de Sistemas Operativos: pipes nominales para comunicación entre procesos, hilos `POSIX` para concurrencia dentro del Monitor, sincronización mediante mutex y variables de condición para controlar el buffer compartido, y semáforos `POSIX` nombrados para coordinar la escritura concurrente de los agentes sobre el pipe.

3. Objetivos

3.1. Objetivo general

Diseñar e implementar un sistema concurrente en C que permita recibir, almacenar y procesar mediciones meteorológicas enviadas por múltiples agentes hacia un monitor mediante comunicación por pipe nominal y procesamiento con hilos `POSIX`.

3.2. Objetivos específicos

- Implementar un proceso Monitor capaz de crear y leer un pipe nominal.
- Implementar procesos Agente capaces de leer archivos `.cvs` y enviar sus lecturas al Monitor.
- Incorporar un buffer compartido de tamaño configurable mediante la bandera `-b`.
- Crear un Hilo Recolector encargado de leer el pipe y depositar las mediciones en el buffer.
- Crear un Hilo Procesador encargado de retirar mediciones del buffer y calcular resultados.
- Sincronizar el acceso al buffer mediante mutex y variables de condición.
- Permitir la ejecución concurrente de varios agentes utilizando el mismo pipe nominal.
- Generar un archivo consolidado con las mediciones recibidas.
- Calcular estadísticas de humedad, rocío y presión.
- Clasificar las lecturas en categorías meteorológicas: lluvioso, nublado, fresco o sin categoría.

4. Estructura del proyecto

El proyecto se organizó en carpetas para separar el código fuente, los archivos de prueba y los archivos de configuración.

Listing 1: Estructura general del proyecto

```
1  .
2      docs
3          archivo-consolidado.cvs
4          documentoTexto2.cvs
5          documentoTexto3.cvs
6          documentoTexto.cvs
7          fresco.cvs
8          lluvioso.cvs
9          mixto.cvs
10         nublado.cvs
11         sin_categoria.cvs
12         varias_estaciones.cvs
13     LICENSE
14     Makefile
15     README.md
16     src
17         agenteMediciones.c
```

```

18     agenteMediciones.h
19     estacion.h
20     monitor.c
21     monitor.h

```

La carpeta **src** contiene los archivos fuente y de cabecera del proyecto. La carpeta **docs** contiene los archivos de mediciones utilizados como entrada y el archivo consolidado generado por el Monitor.

5. Diseño general de la solución

La solución se diseñó con dos programas principales:

- **Monitor:** proceso principal del sistema. Se encarga de crear el pipe nominal, inicializar el buffer, crear los hilos, recibir mediciones, procesarlas y generar resultados.
- **Agente:** proceso encargado de leer un archivo **.cvs** y enviar sus lecturas al Monitor a través del pipe nominal.

5.1. Flujo general del sistema

1. El usuario ejecuta primero el Monitor indicando el tamaño del buffer y el nombre del pipe.
2. El Monitor crea un pipe nominal en **/tmp**.
3. El Monitor crea un buffer circular con capacidad para **tamBuffer** lecturas.
4. El Monitor crea el Hilo Recolector y el Hilo Procesador.
5. Uno o varios agentes leen archivos **.cvs** desde la carpeta **docs**.
6. Cada agente envía sus lecturas al pipe nominal.
7. El Hilo Recolector recibe las lecturas y las inserta en el buffer.
8. El Hilo Procesador extrae lecturas del buffer, actualiza estadísticas, clasifica las mediciones y escribe el archivo consolidado.
9. Al finalizar, el Monitor imprime por terminal el resumen estadístico y la categorización meteorológica.

Componente	Función	Mecanismo usado
Agente	Lee archivo y envía mediciones	<code>fopen</code> , <code>write</code> , pipe nominal
Monitor	Recibe y procesa mediciones	<code>mkfifo</code> , <code>pthread</code>
Hilo Recolector	Lee del pipe y llena buffer	<code>fgets</code> , buffer circular
Hilo Procesador	Consume buffer y calcula resultados	<code>mutex</code> , <code>cond variables</code>
Semáforo	Sincroniza agentes	<code>sem_open</code> , <code>sem_wait</code> , <code>sem_post</code>

6. Formato de los archivos de entrada

Los archivos de entrada se almacenan en la carpeta `docs` y tienen extensión `.cvs`. Cada línea representa una medición de una estación meteorológica.

Listing 2: Formato de una línea de medición

```
nombreEstacion,humedad,rocio,presion,hora
```

Ejemplo:

Listing 3: Ejemplo de archivo de mediciones

```
EK,90,9,750,08:00:00
EK,90,9,750,09:01:00
EK,89,8,749,10:00:00
EK,79,4,720,11:00:00
.
```

El carácter `.` indica el final del archivo. Cuando el agente encuentra esta línea, detiene la lectura y no la envía al Monitor.

7. Estructura de datos principal

La estructura `Estacion` se separó en el archivo `estacion.h` para que pudiera ser reutilizada tanto por el Monitor como por el Agente.

Listing 4: Estructura Estacion en estacion.h

```
1  #ifndef ESTACION_H
2  #define ESTACION_H
3
4  #define MAX_NOMBRE_ESTACION 50
5  #define MAX_HORA 20
6  #define MAX_ESTACIONES 100
7
8  typedef struct {
9      char nombreEstacion[MAX_NOMBRE_ESTACION];
10     int humedad;
11     int rocio;
12     int presion;
13     char hora[MAX_HORA];
14 } Estacion;
15
16 #endif
```

Esta estructura representa una lectura individual de una estación. Contiene el nombre de la estación, los valores meteorológicos y la hora asociada a la medición.

8. Comunicación mediante pipe nominal

Un pipe nominal permite comunicar procesos independientes mediante una ruta en el sistema de archivos. En este proyecto, el Monitor crea el pipe nominal en la carpeta `/tmp`, usando el nombre recibido con la bandera `-p`.

Por ejemplo, si se ejecuta:

```
1 ./monitor -b 2 -p ola
```

el Monitor crea el pipe:

```
1 /tmp/ola
```

Los agentes deben usar el mismo nombre de pipe para enviar sus lecturas:

```
1 ./agente -f lluvioso.csv -t 1 -p ola
```

8.1. Creación del pipe nominal

Listing 5: Creación del pipe nominal

```
1 int crearPipeNominal(const char *rutaPipe) {  
2     if (mkfifo(rutaPipe, 0666) == -1) {  
3         if (errno != EEXIST) {  
4             perror("Error creando pipe nominal");  
5             return -1;  
6         }  
7     }  
8  
9     return 0;  
10 }
```

El permiso 0666 indica que el pipe puede ser leído y escrito por el propietario, el grupo y otros usuarios. La condición `errno != EEXIST` permite que el programa no falle si el pipe ya existía de una ejecución anterior.

9. Implementación del Agente

El Agente se encarga de leer un archivo de mediciones y enviar cada línea al Monitor. Su ejecución sigue la forma:

```
1 ./agente -f archivo.csv -t tiempo -p nombre_pipe
```

Donde:

- `-f`: indica el archivo `.csv` a leer.
- `-t`: indica el tiempo de espera entre una lectura y otra.
- `-p`: indica el nombre del pipe nominal creado por el Monitor.

9.1. Lectura automática desde la carpeta docs

Para facilitar la ejecución, el agente construye automáticamente la ruta del archivo dentro de la carpeta docs. Si el usuario ejecuta:

```
1 ./agente -f lluvioso.cvs -t 1 -p ola
```

el programa lee realmente:

```
1 docs/lluvioso.cvs
```

La función encargada de esto es:

Listing 6: Construcción de ruta hacia docs

```
1 void construirRutaArchivo(const char *archivoEntrada, char *rutaArchivo, int
   tamRuta) {
2     if (strchr(archivoEntrada, '/') != NULL) {
3         snprintf(rutaArchivo, tamRuta, "%s", archivoEntrada);
4     } else {
5         snprintf(rutaArchivo, tamRuta, "%s/%s", CARPETA_DOCS, archivoEntrada);
6     }
7 }
```

Si el usuario ya envía una ruta como docs/lluvioso.cvs o /home/usuario/archivo.cvs, la función respeta esa ruta. Si solo envía el nombre del archivo, se antepone automáticamente docs/.

9.2. Envío de lecturas por pipe

Cada lectura se convierte nuevamente en una línea de texto y se envía al pipe con `write`.

Listing 7: Envío de lecturas por pipe desde el Agente

```
1 for (int i = 0; i < cantidad; i++) {
2     char mensaje[MAX_LINEA];
3
4     snprintf(
5         mensaje,
6         sizeof(mensaje),
7         "%s,%d,%d,%d,%s\n",
8         estaciones[i].nombreEstacion,
9         estaciones[i].humedad,
10        estaciones[i].rocio,
11        estaciones[i].presion,
12        estaciones[i].hora
13    );
14
15    sem_wait(semaforo);
16    write(fdPipe, mensaje, strlen(mensaje));
17    sem_post(semaforo);
18 }
```

```

19     if (i < cantidad - 1) {
20         sleep(tiempoSegundos);
21     }
22 }

```

El llamado a `sleep(tiempoSegundos)` permite que el agente espere el tiempo indicado por la bandera `-t` antes de enviar la siguiente lectura.

10. Sincronización entre agentes con semáforo nombrado

Como pueden existir varios agentes escribiendo al mismo pipe nominal, se incorporó un semáforo nombrado POSIX. Este semáforo funciona como un candado compartido entre procesos.

Listing 8: Uso del semáforo para proteger la escritura al pipe

```

1 sem_wait(semaforo);
2 write(fdPipe, mensaje, strlen(mensaje));
3 sem_post(semaforo);

```

El objetivo de este bloque es garantizar que solo un agente escriba en el pipe en un instante determinado. Aunque las escrituras pequeñas sobre pipes pueden ser atómicas, el uso del semáforo hace explícita la sincronización entre procesos, cumpliendo con el propósito del proyecto de aplicar mecanismos de coordinación concurrente.

El Monitor crea el semáforo con:

```

1 sem_t *semaforo = sem_open(NOMBRE_SEMAFORO, O_CREAT, 0666, 1);

```

El valor inicial 1 indica que el semáforo se comporta como un semáforo binario, permitiendo la entrada de un solo agente a la región crítica de escritura.

11. Implementación del Monitor con hilos

El Monitor recibe la entrada:

```

1 ./monitor -b tamBuffer -p pipeNom

```

Donde:

- `-b`: indica que el siguiente valor corresponde al tamaño del buffer.
- `tamBuffer`: representa cuántas lecturas puede almacenar temporalmente el buffer.
- `-p`: indica el nombre del pipe nominal.
- `pipeNom`: es el nombre del pipe usado por los agentes para enviar datos.

11.1. Hilos del Monitor

El Monitor utiliza dos hilos principales:

- **Hilo Recolector:** lee las líneas recibidas por el pipe nominal, las convierte a estructuras `Estacion` y las inserta en el buffer compartido.
- **Hilo Procesador:** retira las estructuras `Estacion` del buffer, escribe el archivo consolidado, actualiza estadísticas y clasifica las lecturas.

Listing 9: Creación de hilos en el Monitor

```
1 pthread_t recolector;  
2 pthread_t procesador;  
3  
4 pthread_create(&procesador, NULL, hiloProcesador, &contexto);  
5 pthread_create(&recolector, NULL, hiloRecolector, &contexto);  
6  
7 pthread_join(recolector, NULL);  
8 pthread_join(procesador, NULL);
```

Se crea primero el Hilo Procesador para que esté esperando datos en el buffer. Luego se crea el Hilo Recolector, que comienza a recibir lecturas desde el pipe nominal.

12. Buffer compartido productor-consumidor

El buffer compartido se implementó como un buffer circular. Su tamaño depende del valor recibido con la bandera `-b`.

Listing 10: Estructura del buffer compartido

```
1 typedef struct {  
2     Estacion *datos;  
3     int capacidad;  
4     int inicio;  
5     int fin;  
6     int cantidad;  
7     int terminado;  
8  
9     pthread_mutex_t mutex;  
10    pthread_cond_t noLleno;  
11    pthread_cond_t noVacio;  
12 } BufferEstaciones;
```

Los campos `inicio` y `fin` permiten tratar el arreglo como circular. El campo `cantidad` indica cuántas lecturas hay actualmente en el buffer. El campo `terminado` indica que el Hilo Recolector ya no insertará más datos.

12.1. Inserción en el buffer

El Hilo Recolector inserta datos en el buffer. Si el buffer está lleno, debe esperar.

Listing 11: Inserción sincronizada en el buffer

```
1 void insertarBuffer(BufferEstaciones *buffer, Estacion estacion) {
2     pthread_mutex_lock(&buffer->mutex);
3
4     while (buffer->cantidad == buffer->capacidad) {
5         pthread_cond_wait(&buffer->noLleno, &buffer->mutex);
6     }
7
8     buffer->datos[buffer->fin] = estacion;
9     buffer->fin = (buffer->fin + 1) % buffer->capacidad;
10    buffer->cantidad++;
11
12    pthread_cond_signal(&buffer->noVacio);
13    pthread_mutex_unlock(&buffer->mutex);
14 }
```

12.2. Extracción del buffer

El Hilo Procesador saca datos del buffer. Si el buffer está vacío, debe esperar.

Listing 12: Extracción sincronizada del buffer

```
1 bool sacarBuffer(BufferEstaciones *buffer, Estacion *estacion) {
2     pthread_mutex_lock(&buffer->mutex);
3
4     while (buffer->cantidad == 0 && !buffer->terminado) {
5         pthread_cond_wait(&buffer->noVacio, &buffer->mutex);
6     }
7
8     if (buffer->cantidad == 0 && buffer->terminado) {
9         pthread_mutex_unlock(&buffer->mutex);
10        return false;
11    }
12
13    *estacion = buffer->datos[buffer->inicio];
14    buffer->inicio = (buffer->inicio + 1) % buffer->capacidad;
15    buffer->cantidad--;
16
17    pthread_cond_signal(&buffer->noLleno);
18    pthread_mutex_unlock(&buffer->mutex);
19
20    return true;
21 }
```

Esta lógica corresponde al problema clásico productor-consumidor. El Hilo Recolector actúa como productor y el Hilo Procesador como consumidor.

13. Procesamiento de mediciones

Cada línea recibida por el Monitor se convierte a una estructura `Estacion`. Para esto se usa `strtok`, separando los campos por comas.

Listing 13: Conversión de línea CSV a estructura `Estacion`

```
1 int convertirLineaAEstacion(const char *linea, Estacion *estacion) {
2     char copia[MAX_LINEA];
3
4     strncpy(copia, linea, sizeof(copia));
5     copia[sizeof(copia) - 1] = '\0';
6
7     char *nombre = strtok(copia, ",");
8     char *humedad = strtok(NULL, ",");
9     char *rocio = strtok(NULL, ",");
10    char *presion = strtok(NULL, ",");
11    char *hora = strtok(NULL, ",");
12
13    if (nombre == NULL || humedad == NULL || rocio == NULL || presion == NULL ||
14    hora == NULL) {
15        return 0;
16    }
17
18    strncpy(estacion->nombreEstacion, nombre, sizeof(estacion->nombreEstacion));
19    estacion->humedad = atoi(humedad);
20    estacion->rocio = atoi(rocio);
21    estacion->presion = atoi(presion);
22    strncpy(estacion->hora, hora, sizeof(estacion->hora));
23
24    return 1;
25 }
```

Si una línea no contiene todos los campos requeridos, se considera inválida y no se procesa.

14. Cálculo de estadísticas

El Monitor calcula estadísticas acumuladas para las variables:

- Humedad.
- Rocío.
- Presión.

Para cada variable se calcula:

- Suma acumulada.
- Promedio.
- Valor mínimo.
- Valor máximo.

El promedio se calcula como:

$$\text{promedio} = \frac{\text{suma de valores}}{\text{cantidad de lecturas}}$$

La estructura utilizada para almacenar estas estadísticas es:

Listing 14: Estructura de estadísticas

```
1 typedef struct {  
2     int cantidad;  
3  
4     int sumaHumedad;  
5     int sumaRocio;  
6     int sumaPresion;  
7  
8     int minHumedad;  
9     int maxHumedad;  
10  
11    int minRocio;  
12    int maxRocio;  
13  
14    int minPresion;  
15    int maxPresion;  
16 } Estadisticas;
```

15. Reglas de categorización meteorológica

Las lecturas se clasifican según reglas definidas sobre humedad, rocío y presión.

Categoría	Humedad	Rocío	Presión
Lluvioso	> 90	> 9	< 750
Nublado	$80 \leq h \leq 95$	> 8	$= 751$
Fresco	< 80	$5 \leq r \leq 8$	> 754

Si una lectura no cumple ninguna regla, se clasifica como `sin categoría`.

Listing 15: Clasificación de una lectura

```

1 void clasificarLectura(Estacion estacion, ConteoCategorias *conteo) {
2     if (estacion.humedad > 90 && estacion.rocio > 9 && estacion.presion < 750) {
3         conteo->lluvioso++;
4         return;
5     }
6
7     if (estacion.humedad >= 80 && estacion.humedad <= 95 &&
8         estacion.rocio > 8 && estacion.presion == 751) {
9         conteo->nublado++;
10        return;
11    }
12
13    if (estacion.humedad < 80 && estacion.rocio >= 5 &&
14        estacion.rocio <= 8 && estacion.presion > 754) {
15        conteo->fresco++;
16        return;
17    }
18
19    conteo->sinCategoria++;
20 }

```

Inicialmente se evaluó la posibilidad de categorizar usando promedios generales. Sin embargo, se encontró que los promedios pueden mezclar condiciones distintas y producir valores intermedios que no cumplen ninguna regla. Por esta razón, se decidió clasificar cada lectura individualmente y luego calcular la categoría predominante.

16. Archivo consolidado

El Monitor genera el archivo:

```
1 docs/archivo-consolidado.cvs
```

Este archivo contiene las lecturas procesadas por el Hilo Procesador.

Listing 16: Ejemplo de archivo consolidado

```

tipo,nombreEstacion,humedad,rocio,presion,hora
LECTURA,EK,90,9,750,08:00:00
LECTURA,EK,90,9,750,09:01:00
LECTURA,EK,89,8,749,10:00:00
LECTURA,EK,79,4,720,11:00:00

```

Los cálculos estadísticos no se escriben en este archivo. Estos se imprimen únicamente por terminal para separar los datos crudos de los resultados calculados.

17. Compilación con Makefile

El proyecto se compila mediante un **Makefile**. Este archivo permite compilar automáticamente el Monitor y el Agente, ubicando los archivos fuente dentro de la carpeta **src**.

Listing 17: Compilación del proyecto

```
1 make
```

También se puede limpiar la compilación con:

```
1 make clean
```

Una versión simplificada del Makefile es:

Listing 18: Makefile del proyecto

```
1 CC = gcc
2
3 SRC_DIR = src
4 BUILD_DIR = build
5
6 CFLAGS = -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -I$(SRC_DIR) -pthread
7 DOCS_DIR = docs
8
9 AGENTE_SRC = $(SRC_DIR)/agenteMediciones.c
10 MONITOR_SRC = $(SRC_DIR)/monitor.c
11
12 AGENTE_OBJ = $(BUILD_DIR)/agenteMediciones.o
13 MONITOR_OBJ = $(BUILD_DIR)/monitor.o
14
15 AGENTE_BIN = agente
16 MONITOR_BIN = monitor
17
18 all: dirs $(AGENTE_BIN) $(MONITOR_BIN)
19
20 dirs:
21     mkdir -p $(BUILD_DIR)
22
23 $(AGENTE_BIN): $(AGENTE_OBJ)
24     $(CC) $(CFLAGS) $^ -o $@
25
26 $(MONITOR_BIN): $(MONITOR_OBJ)
27     $(CC) $(CFLAGS) $^ -o $@
28
29 $(BUILD_DIR)/agenteMediciones.o: $(AGENTE_SRC) $(SRC_DIR)/agenteMediciones.h
30     $(CC) $(CFLAGS) -DCARPETA_DOCS=\"$(DOCS_DIR)\" -c $< -o $@
31
32 $(BUILD_DIR)/monitor.o: $(MONITOR_SRC) $(SRC_DIR)/monitor.h $(SRC_DIR)/estacion.h
33     $(CC) $(CFLAGS) -c $< -o $@
34
35 clean:
36     rm -rf $(BUILD_DIR) agente monitor
37
38 .PHONY: all dirs clean
```


La opción `-pthread` es necesaria porque el Monitor utiliza hilos POSIX y el proyecto también emplea mecanismos de sincronización asociados al entorno POSIX.

18. Ejecución del sistema

18.1. Ejecución del Monitor

Primero se debe ejecutar el Monitor:

```
1 ./monitor -b 2 -p ola
```

Esto crea un buffer de dos lecturas y un pipe nominal llamado `/tmp/ola`.

18.2. Ejecución de un Agente

En otra terminal se ejecuta un agente:

```
1 ./agente -f lluvioso.cvs -t 1 -p ola
```

Esto envía las lecturas del archivo `docs/lluvioso.cvs`, esperando un segundo entre cada lectura.

18.3. Ejecución concurrente de varios agentes

Para probar concurrencia, se pueden ejecutar varios agentes al mismo tiempo:

```
1 ./agente -f lluvioso.cvs -t 1 -p ola &  
2 ./agente -f fresco.cvs -t 1 -p ola &  
3 ./agente -f nublado.cvs -t 1 -p ola &
```

Todos los agentes escriben en el mismo pipe nominal y el Monitor procesa las lecturas mediante su buffer compartido.

19. Pruebas realizadas y validación del sistema

Para comprobar el correcto funcionamiento del sistema se diseñaron diferentes archivos de prueba en formato `.cvs`, ubicados dentro de la carpeta `docs`. Cada archivo fue construido para validar una condición específica del sistema: lectura de datos, envío por pipe nominal, recepción concurrente, procesamiento con buffer, cálculo estadístico y clasificación meteorológica.

19.1. Archivos de prueba

Archivo	Propósito de la prueba
lluvioso.cvs	Validar lecturas que deben clasificarse como <i>lluvioso</i> .
nublado.cvs	Validar lecturas que deben clasificarse como <i>nublado</i> .
fresco.cvs	Validar lecturas que deben clasificarse como <i>fresco</i> .
mixto.cvs	Probar lecturas de varias categorías en un mismo archivo.
sin_categoria.cvs	Verificar lecturas que no cumplen ninguna regla meteorológica.
varias_estaciones.cvs	Validar mediciones provenientes de diferentes estaciones.
documentoTexto.cvs	Archivo usado para pruebas iniciales de lectura y envío por pipe.
documentoTexto2.cvs	Archivo auxiliar para validar múltiples entradas.
documentoTexto3.cvs	Archivo auxiliar para validar consolidación de datos.

Estas pruebas permiten verificar que el sistema cumpla con los siguientes aspectos funcionales:

- Lectura correcta de archivos `.cvs`.
- Envío de mediciones desde el Agente hacia el Monitor.
- Comunicación mediante pipe nominal.
- Recepción línea por línea en el Hilo Recolector.
- Uso del buffer compartido definido por `-b tamBuffer`.
- Procesamiento de lecturas mediante el Hilo Procesador.
- Sincronización entre hilos mediante mutex y variables de condición.
- Sincronización entre agentes mediante semáforo nombrado.
- Generación del archivo consolidado.
- Cálculo de estadísticas de humedad, rocío y presión.
- Clasificación meteorológica de las lecturas.

19.2. Caso de prueba 1: clasificación lluviosa

Objetivo. Validar que el sistema clasifique correctamente como *lluviosas* las lecturas con humedad mayor a 90, rocío mayor a 9 y presión menor a 750.

Comando del Monitor:

```
1 ./monitor -b 2 -p ola
```

Comando del Agente:

```
1 ./agente -f lluvioso.cvs -t 1 -p ola
```

Entrada usada en docs/lluvioso.cvs:

```
EK,91,10,745,08:00:00
EK,94,11,748,09:00:00
EK,92,12,740,10:00:00
EK,96,10,749,11:00:00
.
```

Resultado esperado:

- El Agente envía cuatro lecturas al pipe nominal.
- El Hilo Recolector recibe las cuatro lecturas.
- El Hilo Procesador las escribe en docs/archivo-consolidado.cvs.
- La categoría predominante reportada por el Monitor es **Lluvioso**.

Listing 19: Salida esperada parcial

```
Lecturas lluviosas: 4
Lecturas nubladas: 0
Lecturas frescas: 0
Lecturas sin categoria: 0
Categoria predominante: Lluvioso
```

19.3. Caso de prueba 2: clasificación fresca

Objetivo. Verificar que el sistema clasifique como *frescas* las lecturas con humedad menor a 80, rocío entre 5 y 8, y presión mayor a 754.

Comando del Agente:

```
1 ./agente -f fresco.cvs -t 1 -p ola
```

Entrada usada en docs/fresco.cvs:

```
EK,70,5,755,08:00:00
EK,75,6,756,09:00:00
EK,79,8,760,10:00:00
EK,68,7,758,11:00:00
.
```

Resultado esperado:

- El Monitor procesa cuatro lecturas.
- Todas las lecturas incrementan el contador de la categoría *fresco*.
- La categoría predominante es **Fresco**.

Listing 20: Salida esperada parcial

```
Lecturas lluviosas: 0
Lecturas nubladas: 0
Lecturas frescas: 4
Lecturas sin categoria: 0
Categoria predominante: Fresco
```

19.4. Caso de prueba 3: archivo mixto

Objetivo. Comprobar que el sistema pueda procesar lecturas de diferentes categorías dentro de un mismo archivo y determinar una categoría predominante.

Comando del Agente:

```
1 ./agente -f mixto.cvs -t 1 -p ola
```

Entrada usada en docs/mixto.cvs:

```
EK,91,10,745,08:00:00
EK,87,10,751,09:00:00
EK,75,6,756,10:00:00
EK,92,11,748,11:00:00
EK,88,9,751,12:00:00
.
```

Resultado esperado:

- El Monitor recibe cinco lecturas.
- Se registran lecturas lluviosas, nubladas y frescas.
- La categoría predominante corresponde a la categoría con mayor conteo.

Listing 21: Salida esperada parcial

```
Lecturas lluviosas: 2
Lecturas nubladas: 2
Lecturas frescas: 1
Lecturas sin categoría: 0
Categoría predominante: Lluvioso
```

En caso de empate, el programa reporta la primera categoría que alcanza el mayor conteo según el orden de evaluación implementado: lluvioso, nublado y fresco.

19.5. Caso de prueba 4: lecturas sin categoría

Objetivo. Validar que el Monitor identifique correctamente las lecturas que no cumplen ninguna de las reglas meteorológicas definidas.

Comando del Agente:

```
1 ./agente -f sin_categoria.cvs -t 1 -p ola
```

Entrada usada en docs/sin_categoria.cvs:

```
EK,87,7,742,08:00:00
EK,89,8,750,09:00:00
EK,80,4,760,10:00:00
EK,96,7,751,11:00:00
.
```

Resultado esperado:

- El Monitor procesa cuatro lecturas.
- Ninguna lectura se clasifica como lluviosa, nublada o fresca.
- Todas las lecturas quedan registradas como `sin categoría`.

Listing 22: Salida esperada parcial

```
Lecturas lluviosas: 0
Lecturas nubladas: 0
Lecturas frescas: 0
Lecturas sin categoria: 4
Categoria predominante: No categorizado
```

19.6. Caso de prueba 5: concurrencia con varios agentes

Objetivo. Validar que varios agentes puedan enviar mediciones al mismo Monitor usando el mismo pipe nominal, mientras el Monitor procesa las lecturas mediante el buffer compartido.

Comando del Monitor:

```
1 ./monitor -b 2 -p ola
```

Comandos de agentes concurrentes:

```
1 ./agente -f lluvioso.csvs -t 1 -p ola &
2 ./agente -f fresco.csvs -t 1 -p ola &
3 ./agente -f nublado.csvs -t 1 -p ola &
```

Resultado esperado:

- El Monitor recibe lecturas provenientes de los tres agentes.
- El Hilo Recolector inserta las lecturas en el buffer circular.
- El Hilo Procesador retira las lecturas del buffer y las escribe en el archivo consolidado.
- El semáforo nombrado sincroniza la escritura de los agentes sobre el pipe nominal.
- El archivo docs/archivo-consolidado.csvs contiene las lecturas recibidas de todos los agentes.

Listing 23: Salida esperada parcial del Monitor

```
Monitor esperando datos en: /tmp/ola
Tamano del buffer: 2 lecturas
[Recolector] Lectura recibida: ...
[Procesador] Procesando: ...
Archivo consolidado creado: docs/archivo-consolidado.csvs
```

19.7. Resumen de validación

Prueba	Funcionalidad validada	Estado esperado
Archivo lluvioso	Clasificación de lecturas lluviosas	Correcto
Archivo fresco	Clasificación de lecturas frescas	Correcto
Archivo mixto	Cálculo de categoría predominante	Correcto
Sin categoría	Manejo de lecturas fuera de regla	Correcto
Varios agentes	Concurrencia, pipe, semáforo y buffer	Correcto

20. Resultados esperados

Al finalizar la ejecución de los agentes, el Monitor imprime por terminal un resumen similar al siguiente:

Listing 24: Ejemplo de salida estadística del Monitor

```
===== RESUMEN DE MEDICIONES =====
Cantidad de lecturas procesadas: 4

Humedad:
  Promedio: 87.00
  Minimo: 79
  Maximo: 90

Rocio:
  Promedio: 7.50
  Minimo: 4
  Maximo: 9

Presion:
  Promedio: 742.25
  Minimo: 720
  Maximo: 750
=====
```

También imprime el resumen de categorización:

Listing 25: Ejemplo de categorización meteorológica

```
===== CATEGORIZACION METEOROLGICA =====
Lecturas lluviosas: 2
Lecturas nubladas: 1
Lecturas frescas: 1
Lecturas sin categoria: 0

Categora predominante: Lluvioso
=====
```

21. Problemas encontrados y decisiones de diseño

Durante el desarrollo se encontraron varios puntos relevantes:

21.1. Lectura del pipe con read

Inicialmente se utilizó `read` para recibir datos desde el pipe. Sin embargo, se observó que una sola lectura podía traer varias líneas juntas, ya que el pipe funciona como un flujo de bytes y no como una estructura de mensajes separados.

Por esta razón se cambió a `fdopen` y `fgets`, permitiendo procesar las mediciones línea por línea.

21.2. Rutas relativas hacia docs

Se identificó que la ruta `docs/archivo.cvs` depende del directorio desde donde se ejecuta el programa, no del lugar donde se encuentra el archivo `.c`. Por esta razón, se decidió ejecutar el proyecto desde la raíz del repositorio.

21.3. Categorización mediante promedios

También se evaluó categorizar usando promedios generales. Esta estrategia podía generar resultados no categorizados cuando las lecturas pertenecían a condiciones distintas. Por ello se adoptó la clasificación por lectura individual y luego se reporta la categoría predominante.

21.4. Uso de hilos en el Monitor

El enunciado menciona explícitamente un Hilo Recolector, por lo que se incorporaron hilos POSIX en el Monitor. El Agente no requiere hilos, ya que cada ejecución del agente corresponde a un proceso independiente.

22. Conclusiones

El proyecto permitió aplicar de forma integrada varios conceptos fundamentales de Sistemas Operativos. En primer lugar, se implementó comunicación entre procesos independientes mediante un pipe nominal, permitiendo que varios agentes enviaran información a un mismo monitor.

En segundo lugar, se implementó concurrencia dentro del Monitor mediante hilos POSIX. El uso de un Hilo Recolector y un Hilo Procesador permitió separar la recepción de datos del procesamiento de las mediciones. Esta división mejora la organización del programa y corresponde al esquema productor-consumidor.

El buffer compartido de tamaño configurable permitió representar una zona crítica real dentro del Monitor. Para manejarla correctamente se utilizaron mutex y variables de condición, evitando condiciones de carrera y permitiendo que los hilos esperaran cuando el buffer estuviera lleno o vacío.

Además, el uso de semáforos nombrados en los agentes permitió sincronizar la escritura concurrente sobre el pipe nominal. Esto resulta útil cuando varios procesos intentan enviar mediciones al mismo tiempo.

Finalmente, el sistema no solo transmite información, sino que también la procesa. El Monitor genera un archivo consolidado, calcula estadísticas de humedad, rocío y presión, y clasifica las mediciones según reglas meteorológicas. De esta manera, el proyecto integra comunicación, concurrencia, sincronización y procesamiento de datos en una solución funcional.

23. Referencias

Referencias

- [1] Tanenbaum, A. S. (2009). *Modern Operating Systems*. Pearson.
- [2] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts*. Wiley.
- [3] Stallings, W. (2018). *Operating Systems: Internals and Design Principles*. Pearson.

- [4] The Open Group. (2018). *The Open Group Base Specifications Issue 7, 2018 Edition*. Disponible en: <https://pubs.opengroup.org/onlinepubs/9699919799.2018edition/>
- [5] The Open Group. (2018). *pthread_create — The Open Group Base Specifications*. Disponible en: https://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_create.html
- [6] The Open Group. (2018). *pthread_join — The Open Group Base Specifications*. Disponible en: https://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_join.html
- [7] Linux Manual Pages. (s.f.). *mkfifo(3) — Linux manual page*. Disponible en: <https://man7.org/linux/man-pages/man3/mkfifo.3.html>
- [8] Linux Manual Pages. (s.f.). *sem_open(3) — Linux manual page*. Disponible en: https://man7.org/linux/man-pages/man3/sem_open.3.html